CRM DE ATENDIMENTO E ORÇAMENTOS

Entendendo o Backend

Parte 3 — Persistência

Ligando o banco de dados sem mudar uma linha dos casos de uso

1 2 3 4 5

Parte 3 de 5 do backend

VISÃO GERAL

Onde estamos no plano do backend

1

Fundação + Domínio

Regras do negócio em Python puro, com testes.

✓ CONCLUÍDA

2

Casos de uso

As ações do usuário, orquestrando o domínio.

✓ CONCLUÍDA

3

Persistência

Banco de dados, tabelas e migrations.

VOCÊ ESTÁ AQUI

4

API REST

Rotas FastAPI que o frontend vai chamar.

5

Autenticação

Login com JWT e gestão de usuários.

Até a Parte 2, tudo vivia na memória: desligou o programa, os dados sumiram. A Parte 3 liga um banco de dados de verdade.

A grande prova da arquitetura: os 8 casos de uso da Parte 2 não mudaram nada. Só foi preciso escrever um novo “plugue”.

CONCEITO

As três peças da persistência

1 PostgreSQL

O banco

Guarda os dados em tabelas, mesmo com o programa desligado. Também protege a integridade: chaves estrangeiras, CHECK, UNIQUE.

2 SQLAlchemy

O tradutor (ORM)

Converte objetos Python em linhas de tabela e vice-versa. Escrevemos Python; ele gera o SQL.

3 Alembic

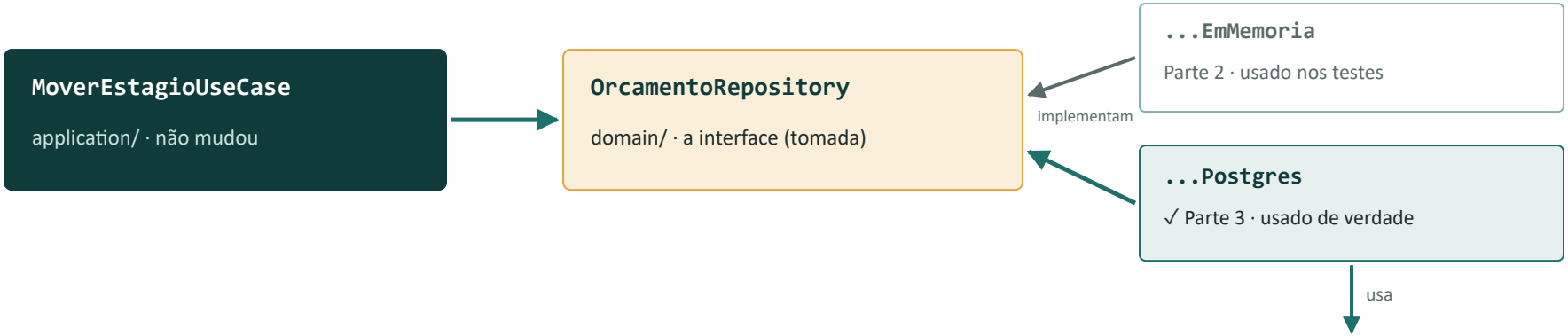
O histórico do esquema

Cria e evolui as tabelas por versões numeradas (migrations), como um “git” da estrutura do banco.

E o domínio? Continua sem saber que o banco existe. Orcamento, Cliente e Telefone não importam nada de SQLAlchemy — o teste de arquitetura da Parte 1 continua vigiando isso.

O DESENHO

O novo plugue e onde cada peça mora



Na hora de rodar, alguém escolhe qual plugue conectar: nos testes de unidade, o em memória; na aplicação (Parte 4), o Postgres. O caso de uso nem percebe a troca.

A REGRA MAIS IMPORTANTE

Model ORM ≠ entidade do domínio

infra/db/models.py — o formato da tabela

```
class OrcamentoModel(Base):
    __tablename__ = "orcamento"

    id: Mapped[UUID] = mapped_column(Uuid, primary_key=True)
    cliente_id: Mapped[UUID] = mapped_column(
        ForeignKey("cliente.id", ondelete="RESTRICT"))
    estagio: Mapped[str] = mapped_column(String(20))
    valor: Mapped[Decimal | None] = mapped_column(Numeric(12, 2))
```

domain/entities/orcamento.py — as regras

```
class Orcamento:
    estagio: EstagioAtendimento     # enum, não texto
    valor: ValorOrcado | None       # Value Object

    def mover_estagio(self, novo, hoje): ...
    def definir_valor(self, valor): ...
    def registrar_interacao(self, ...): ...
```

Model Fala a língua do banco: tipos SQL, chaves, índices. Nenhuma regra de negócio.

Entidade Fala a língua do negócio: estágios, guardas, Value Objects. Nenhum SQL.

Reaproveitar o model como entidade é o atalho que quebra a arquitetura inteira (doc 13). Quem traduz de um para o outro é o repositório.

O ESQUEMA

As quatro tabelas do diagrama ER

cliente

```
id UUID PK
nome
telefone (só dígitos)
email
empresa ?
tipo_embalagem_interesse ?
criado_em com fuso
```

1 — N

RESTRICT

orcamento

```
id UUID PK
cliente_id FK
estagio CHECK (5 valores)
valor NUMERIC(12,2) ?
data_abertura
data_fechamento ?
```

1 — N

CASCADE

interacao

```
id UUID PK
orcamento_id FK
tipo CHECK (3 valores)
conteudo TEXT
data_hora com fuso
```

usuario

```
id UUID PK
nome
email UNIQUE
senha_hash
perfil CHECK
ativo
```

- RESTRICT: não dá para apagar um cliente que tem orçamentos (RF08).
- CASCADE: apagar um orçamento apaga as interações dele (composição).
- CHECK: o banco recusa estágio, tipo ou perfil fora da lista.
- ? = pode ficar vazio. valor vazio é um estado válido: orçamento ainda sem preço (RF03).
- Usuário e Administrador ficam na mesma tabela, separados pela coluna perfil.

POR DENTRO DO CÓDIGO

OrcamentoRepositoryPostgres: o tradutor

```
def salvar(self, orcamento: Orcamento) -> None:
    model = self._sessao.get(OrcamentoModel, orcamento.id)
    if model is None:                          # 1) é novo? cria
        model = OrcamentoModel(id=orcamento.id)
        self._sessao.add(model)

    model.estagio = orcamento.estagio.value  # 2) copia os campos
    model.valor = orcamento.valor.valor if orcamento.valor else None
    ...
    for interacao in orcamento.interacoes:   # 3) só as interações novas
        if interacao.id not in ja_gravadas:
            model.interacoes.append(InteracaoModel(...))

    self._sessao.flush()                       # 4) envia, mas NÃO confirma
```

1 Busca ou cria

Se o id já existe, atualiza; senão, insere. Salvar duas vezes nunca duplica.

2 Só acrescenta

Interações são imutáveis: nunca há update nem delete delas.

3 flush ≠ commit

flush manda o SQL; quem confirma (commit) é a transação da requisição.

O caminho de volta é _para_entidade(): monta um Orcamento a partir do model, reconstruindo EstagioAtendimento, ValorOrcado e as Interações.

RNF08

O banco como última linha de defesa

CHECK **Estágio fora do funil**

O teste tenta: INSERT com estagio = 'QUALQUER'

✗ recusado

RESTRICT **Apagar cliente com histórico**

O teste tenta: DELETE de um cliente com orçamento

✗ recusado

CASCADE **Apagar orçamento**

O teste tenta: DELETE de um orçamento com interações

✓ interações apagadas

UNIQUE **E-mail de usuário repetido**

O teste tenta: dois usuários com a@crm.com

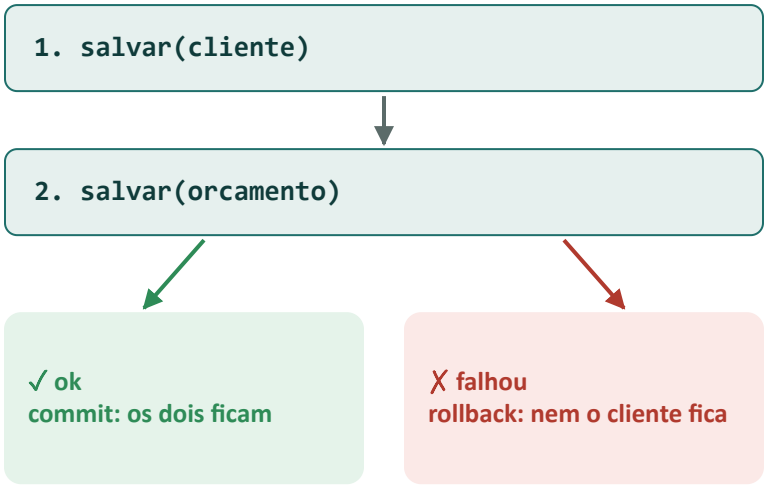
✗ recusado

Os testes fazem escritas diretas no banco, sem passar pelo domínio. Mesmo assim, as regras se mantêm. A regra de negócio continua no domínio; o banco só impede lixo.

Uma transação por requisição

```
@contextmanager
def sessao_transacional(fabrica):
    sessao = fabrica()
    try:
        yield sessao        # o caso de uso trabalha aqui
        sessao.commit()     # deu tudo certo: confirma
    except BaseException:
        sessao.rollback()   # algo falhou: desfaz TUDO
        raise
    finally:
        sessao.close()
```

UC01 grava duas coisas:

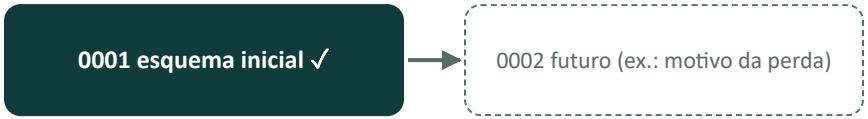


O teste que prova:

um repositório de orçamento “quebrado de propósito” lança erro na 2ª gravação. Depois, abrindo uma sessão nova, o banco tem 0 clientes e 0 orçamentos. Na Parte 4, cada requisição HTTP vai rodar dentro de um sessao_transacional.

Migrations: o histórico da estrutura do banco

Uma migration é um arquivo numerado que diz como criar (upgrade) e desfazer (downgrade) uma mudança nas tabelas.




```
# cria/atualiza as tabelas no banco
python -m alembic upgrade head

# desfaz tudo
python -m alembic downgrade base

# só mostra o SQL, sem precisar de banco
python -m alembic upgrade head --sql
```

SQL gerado para o PostgreSQL (trecho)

```
CREATE TABLE orcamento (
  id UUID NOT NULL,
  cliente_id UUID NOT NULL,
  estagio VARCHAR(20) NOT NULL,
  valor NUMERIC(12, 2),
  data_abertura DATE NOT NULL,
  data_fechamento DATE,
  PRIMARY KEY (id),
  CHECK (estagio IN ('NOVO_CONTATO', ...)),
  FOREIGN KEY(cliente_id) REFERENCES cliente (id)
    ON DELETE RESTRICT
);
```

Um teste garante que a migration e os models nunca divergem: aplica a migration e compara o banco resultante com o models.py.

Um teste, três implementações

```
@pytest.fixture(params=["memoria", "sqlite", "postgres"])
def repos(request): ...


def test_alteracao_sem_salvar_nao_persiste(repos):
    orcamento = cliente_salvo(repos).abrir_orcamento(...)
    repos.orcamentos.salvar(orcamento)

    carregado = repos.orcamentos.buscar_por_id(orcamento.id)
    carregado.mover_estagio(EM_NEGOCIACAO, hoje=HOJE)

    de_novo = repos.orcamentos.buscar_por_id(orcamento.id)
    assert de_novo.estagio is NOVO_CONTATO
```

O mesmo teste roda 3 vezes:

em memória
✓ passa

SQLite
✓ passa

PostgreSQL
pulado: sem TEST_DATABASE_URL

12 cenários × 3 implementações

Se um repositório se comportar diferente dos outros, o teste aponta qual.

É isso que dá confiança para trocar o repositório em memória (dos testes de unidade) pelo Postgres (da aplicação): os dois cumprem o mesmo contrato.

DETALHES QUE FAZEM DIFERENÇA

Dois cuidados para o SQLite se comportar como o Postgres

`DataHoraUTC`

O SQLite não guarda fuso horário

Sem cuidado, 14:30 em Brasília voltaria do banco como “14:30 sem fuso”, e a comparação com o domínio falharia. O tipo DataHoraUTC grava sempre em UTC e devolve sempre com fuso.

`PRAGMA foreign_keys`

O SQLite ignora chaves estrangeiras

...a menos que essa opção seja ligada em cada conexão. Sem ela, RESTRICT e CASCADE não funcionariam nos testes. O session.py liga automaticamente.

Por que não usar só SQLite? Porque a produção é PostgreSQL. O SQLite deixa os testes rodarem em qualquer máquina; as variantes PostgreSQL dos mesmos testes são a validação final.

BASTIDORES

O que travou no caminho e como foi contornado

Problema	Solução
O Smart App Control do Windows bloqueou a parte compilada (DLL) do SQLAlchemy.	SQLAlchemy reinstalado em Python puro: DISABLE_SQLALCHEMY_CEXT=1 pip install --no-binary sqlalchemy sqlalchemy.
Não há PostgreSQL nem Docker, e o driver psycopg não existe para Python 32 bits.	Testes rodam em SQLite agora; as variantes PostgreSQL ficam prontas e rodam quando TEST_DATABASE_URL existir.
Num teste, os índices recém-criados “sumiam”.	Uma conexão aberta antes da migration guardava o esquema antigo em cache. Solução: engine.dispose() depois de preparar o banco.

Pendente: rodar a suíte contra um PostgreSQL real. Precisa de Python 3.12/3.13 de 64 bits e Docker Desktop (ou PostgreSQL instalado). O resto da Parte 3 está pronto e testado.

RESULTADO

Como a Parte 3 terminou

188

testes passando

em cerca de 1,3 segundo

23

testes pulados

as variantes PostgreSQL

4

tabelas criadas

pela migration 0001

0

casos de uso alterados

a arquitetura funcionou

Critério de pronto: pytest verde, mypy --strict e ruff limpos, e o SQL da migration gerado para o PostgreSQL conferido contra o DER. Falta só a execução em um PostgreSQL real.

MÃO NA MASSA

Como explorar você mesmo

```
cd backend

# só os testes de integração, com o nome de cada um
.venv\Scripts\python.exe -m pytest tests/integration -v

# o SQL que a migration gera para o PostgreSQL
set DATABASE_URL=postgresql+psycopg://crm:crm@localhost/crm
.venv\Scripts\python.exe -m alembic upgrade head --sql
```

Ordem sugerida de leitura

1. **`infra/db/models.py`**
   as tabelas
2. **`orcamento_repository_postgres.py`**
   o tradutor
3. **`infra/db/session.py`**
   a transação
4. **`0001_esquema_inicial.py`**
   a migration

Exercício para fixar

1. Em orcamento_repository_postgres.py, comente o laço for que acrescenta as interações em salvar().
2. Rode pytest tests/integration -v. O round-trip falha no SQLite mas passa em memória: o teste de contrato pegou um plugue que não cumpre o combinado.
3. Desfaça a mudança (Ctrl+Z) e rode de novo: tudo verde.

PARA REVISAR

Glossário rápido e próximo passo

ORM	Tradutor objeto ↔ tabela (SQLAlchemy).
Model	Classe que descreve uma tabela. Não é entidade.
Migration	Versão numerada da estrutura do banco (Alembic).
Transação	Grupo de gravações que vale inteiro ou não vale nada.
commit / rollback	Confirmar / desfazer a transação.
Constraint	Regra no banco: FK, CHECK, UNIQUE.

PRÓXIMO PASSO

Parte 4 — API REST

- Rotas FastAPI do contrato da documentação.
- Schemas Pydantic de entrada e saída em camelCase.
- Erros de domínio viram 422; não encontrado vira 404.
- Cada requisição dentro de um sessao_transacional.
- Pendente da Parte 3: validar no PostgreSQL real.